

Configuration Management

Ansible, Configuration vs Provisioning, Agent vs Agentless Models, and Automated Workflows

A conceptual and practical reference for understanding how configuration management fits alongside infrastructure provisioning tools like Terraform.

Contents

- 1. What is Configuration Management?
- 2. Ansible — Core Concepts
- 3. Configuration vs Provisioning
- 4. Agent vs Agentless Models
- 5. Automated Configuration Workflows
- 6. Summary Cheat Sheet

1. What is Configuration Management?

Configuration Management (CM) is the practice of keeping servers, applications, and systems in a known, consistent, desired state — automatically, and repeatably — instead of manually installing packages, editing config files, or applying settings by hand on each machine.

The problem CM solves

- Manually configuring 1 server is easy. Manually configuring 200 servers identically, and keeping them identical over time, is not.
- Without CM, servers drift apart over time (“configuration drift”) as different engineers make different manual changes.
- CM tools describe the desired end state (e.g. “nginx must be installed, running, and enabled on boot”) and repeatedly enforce it.

Common Configuration Management tools

Tool	Model	Language
Ansible	Agentless (SSH/WinRM)	YAML
Puppet	Agent-based	Puppet DSL (Ruby-like)
Chef	Agent-based	Ruby DSL
SaltStack	Agent-based (or agentless via SSH)	YAML

2. Ansible — Core Concepts

Ansible is an open-source, agentless configuration management and automation tool from Red Hat. It connects to remote machines over SSH (or WinRM for Windows) and runs tasks defined in simple YAML files.

Key building blocks

Inventory

A file listing the machines Ansible manages, optionally grouped.

```
[webservers]
web1.example.com
web2.example.com

[dbservers]
db1.example.com
```

Playbook

A YAML file describing what should happen on which hosts — the core unit of automation in Ansible.

```
--
- name: Install and start nginx
  hosts: webservers
  become: true
  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present

    - name: Ensure nginx is running
      service:
        name: nginx
        state: started
        enabled: true
```

Modules

Reusable units of work Ansible ships with (apt, yum, service, copy, template, user, and hundreds more). Each module is idempotent — running it twice produces the same end state, not duplicate actions.

Roles

A structured way to package related tasks, variables, templates, and handlers together for reuse across projects.

```
roles/
├── webserver/
│   ├── tasks/main.yml
│   ├── handlers/main.yml
│   ├── templates/
│   ├── vars/main.yml
│   └── defaults/main.yml
```

Running a playbook

```
ansible-playbook -i inventory.ini site.yml
```

Idempotency: The single most important concept in Ansible (and CM generally): running the same playbook 10 times produces the same result as running it once. Ansible checks current state before acting, and only makes changes when something doesn't already match the desired state.

3. Configuration vs Provisioning

These two terms are often confused because tools like Terraform and Ansible are frequently used together, but they solve different problems.

Definitions

- Provisioning = creating the infrastructure itself: the virtual machine, the network, the disk, the load balancer. “Does this server exist?”
- Configuration = setting up what runs inside that infrastructure once it exists: installing packages, writing config files, starting services, creating users. “Is this server set up correctly?”

Side-by-side comparison

Aspect	Provisioning (e.g. Terraform)	Configuration (e.g. Ansible)
Primary question	Does the resource exist?	Is the resource configured correctly?
Typical actions	Create VM, VPC, subnet, disk, DNS record	Install packages, edit config files, manage services
State awareness	Tracks state in a state file	Mostly stateless; checks live system state each run
Runs against	Cloud provider APIs (AWS, Azure, GCP)	The operating system inside a running machine
Typical order	Runs first	Runs after provisioning completes

How they work together in practice

1. Terraform provisions the EC2 instance / Azure VM (creates the compute resource).
2. Terraform outputs the instance's IP address.
3. Ansible takes that IP, adds it to its inventory, and configures the software inside it (installs nginx, deploys the app, sets up users).

Common pattern: Terraform hands off to Ansible using a “local-exec” provisioner, a dynamic inventory plugin that queries AWS/Azure directly, or simply by piping Terraform outputs into an Ansible inventory file in a CI/CD pipeline.

4. Agent vs Agentless Models

This describes HOW a configuration management tool actually reaches and controls a remote machine.

Agent-based model (e.g. Puppet, Chef)

- A small piece of software (the “agent”) is installed and runs continuously on every managed machine.
- The agent periodically checks in with a central server (“master”) to pull the latest desired configuration and applies it locally.
- Requires: agent installation, ongoing agent updates, agent-to-master network connectivity, and a central master server to maintain.

Agentless model (e.g. Ansible)

- No software needs to be pre-installed on managed machines.
- Ansible connects over standard SSH (Linux) or WinRM (Windows) that's already present on virtually every server, runs its instructions, and disconnects.
- Requires only: SSH/WinRM access and Python (or PowerShell) already present on the target — both of which ship with almost every OS by default.

Comparison

Aspect	Agent-based	Agentless
Setup overhead	Install + maintain an agent on every host	None — uses existing SSH/WinRM
Central server required	Yes (master/server)	No (or optional, e.g. Ansible Tower/AWX)
Ongoing checks	Agent polls continuously in the background	Runs only when you execute it
Network requirement	Persistent agent-to-master connection	On-demand SSH/WinRM connection only
Scaling to new hosts	Must install agent first	Add to inventory and run — nothing to install
Resource usage on host	Agent uses CPU/RAM continuously	Zero footprint when not actively running

Trade-off to teach: Agentless tools like Ansible are simpler to start with and have zero footprint, but rely on network access (SSH) being available whenever you want to make a change. Agent-based tools like Puppet/Chef enforce configuration continuously and can self-heal drift automatically even without someone triggering a run — at the cost of needing an agent on every machine.

5. Automated Configuration Workflows

In real environments, configuration management isn't run manually by a person typing `ansible-playbook` — it's triggered automatically as part of a larger pipeline.

A typical end-to-end automated workflow

4. Developer pushes code / infrastructure change to a Git repository.
5. A CI/CD pipeline (e.g. Jenkins, GitHub Actions, GitLab CI, Azure DevOps) is triggered automatically.
6. Terraform stage: `terraform plan` runs and is reviewed (often auto-approved in lower environments); `terraform apply` provisions or updates infrastructure.
7. Dynamic inventory stage: the pipeline queries the cloud provider (or reads Terraform outputs) to get the current list of live server IPs.
8. Ansible stage: `ansible-playbook` runs automatically against that inventory, configuring the newly provisioned (or existing) servers.
9. Validation stage: automated checks (health checks, smoke tests) confirm the servers are configured and the application responds correctly.
10. Notification: the pipeline reports success/failure back to the team (Slack, email, Teams).

Example: GitHub Actions workflow combining both tools

```
name: Provision and Configure
on: [push]
jobs:
  provision:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Terraform Init & Apply
        run: |
          terraform init
          terraform apply -auto-approve
      - name: Export inventory from Terraform output
        run: terraform output -json instance_ips > inventory.json
      - name: Run Ansible playbook
        run: ansible-playbook -i inventory.json site.yml
```

Why automate the workflow at all

- Removes manual, error-prone steps — a person forgetting to run a playbook after provisioning is a common real-world outage cause.
- Makes environments reproducible — spinning up a new environment (e.g. for a new client, or disaster recovery) becomes a single pipeline trigger.
- Enables consistent, auditable change history — every infrastructure and configuration change is a Git commit and a pipeline run, not an undocumented manual SSH session.

Teaching takeaway: Provisioning tools (Terraform) and configuration tools (Ansible) are complementary, not competing. A mature automated workflow chains them together: Terraform builds the house, Ansible furnishes it, and CI/CD is the crew that does both every time, automatically, in the same order, without anyone typing commands by hand.

6. Summary Cheat Sheet

Question	Answer
What does CM solve?	Prevents configuration drift; keeps many servers in a consistent, known state automatically
What is Ansible?	An agentless, YAML-based configuration management and automation tool using SSH/WinRM
Provisioning vs Configuration?	Provisioning creates infrastructure (Terraform); Configuration sets up what runs inside it (Ansible)
Agent vs Agentless?	Agent = software pre-installed on every host, checks in with a master (Puppet/Chef). Agentless = connects on-demand over SSH/WinRM, nothing pre-installed (Ansible)
What is idempotency?	Running the same automation repeatedly produces the same end state, never duplicate or repeated actions
What is a typical automated workflow?	Git push → CI/CD trigger → Terraform provisions → dynamic inventory → Ansible configures → validation → notification

End of Document